

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”

Факультет програмних систем та прикладної математики
Кафедра програмного забезпечення комп’ютерних систем



Лабораторна робота № 2
з дисципліни “Розроблення вебзастосунків”
тема «Робота з постійним сховищем, валідація вхідних даних»

Виконав
студент III курсу
групи КП-42
Жижка Богдан

Київ 2026

Мета роботи: Отримати навички роботи з технологією ORM для постійного збереження сутностей REST API у базі даних, а також використання шаблонів проєктування «Репозиторій» та «Data Transfer Object».

Завдання:

1. Замінити у програмному забезпеченні, розробленому в ході виконання першої роботи комп'ютерного практикуму, тимчасове зберігання даних на постійне у базі даних.
2. Додати валідацію вхідних даних.

ХІД РОБОТИ

1. Постановка задачі Для виконання комп'ютерного практикуму використано предметну галузь з першої роботи — «Система управління таксопарком». Система призначена для обліку автомобілів таксопарку, а також фіксації водіїв, що за ними закріплені. У другій роботі тимчасове зберігання даних у JSON-файлі замінено на постійне у базі даних PostgreSQL. Для роботи з БД використано ORM Entity Framework Core та шаблон «Репозиторій», репозиторії та сервіси отримуються через IoC-контейнер, REST API працює з окремими DTO, а вхідні DTO валідуються за допомогою бібліотеки FluentValidation. Додатково реалізовано асинхронні методи роботи з базою даних та мапінг сутностей у DTO за допомогою AutoMapper.

Сутності мають між собою зв'язок «один-до-багатьох»: один автомобіль може мати багато водіїв, що працюють позмінно, а кожен водій закріплений за одним автомобілем. При видаленні автомобіля його водії видаляються каскадно.

Автомобіль (Car) — сутність, що репрезентує транспортний засіб у системі. Містить наступні властивості:

- Id (ціле число) — унікальний ідентифікатор автомобіля, генерується базою даних.
- Brand (рядок) — марка автомобіля.
- Model (рядок) — модель автомобіля.
- EngineVolume (дійсне число) — об'єм двигуна.
- Year (ціле число) — рік випуску транспортного засобу.
- Age (ціле число) — вік автомобіля, обчислюється під час формування відповіді та в БД не зберігається.

Водій (Driver) — сутність, що зберігає дані про працівника таксопарку. Містить наступні властивості:

- Id (ціле число) — унікальний ідентифікатор водія, генерується базою даних.
- FullName (рядок) — повне ім’я водія.
- Rating (дійсне число) — рейтинг водія в системі оцінювання.
- IsBusy (логічне значення) — поточний статус водія (вільний або виконує замовлення).
- CarId (ціле число) — зовнішній ключ на автомобіль, за яким закріплений водій.

Валідаційні правила. Для створення (CarCreateDto, DriverCreateDto) та оновлення (CarUpdateDto, DriverUpdateDto) сутностей застосовуються однакові правила. Якщо дані некоректні, API повертає код 400 Bad Request зі списком помилок для кожного поля.

Правила для автомобіля:

- Brand — обов’язкове поле, не більше 50 символів.
- Model — обов’язкове поле, не більше 50 символів.
- EngineVolume — значення від 0 до 10 л (0 — для електромобілів).
- Year — рік випуску від 2000 до наступного календарного року.

Правила для водія:

- FullName — обов’язкове поле, від 3 до 100 символів, може містити лише літери, пробіли, дефіс та апостроф.
- Rating — значення від 0 до 5.
- CarId — додатне число; автомобіль з таким ідентифікатором має існувати в базі даних, інакше повертається помилка 400.

Якщо запитаний автомобіль або водій не існує, API повертає код 404 Not Found.

Код розробленого програмного забезпечення

Controllers

```
using Microsoft.AspNetCore.Mvc;
using Taxi.DTOs;
using Taxi.Services;

namespace Taxi.Controllers;

[ApiController]
[Route("api/[controller]")]
[Produces("application/json")]
public class CarsController : ControllerBase
{
    private readonly ICarService carService;

    public CarsController(ICarService carService)
    {
        this.carService = carService;
    }

    [HttpGet]
                                [ProducesResponseType(typeof(List<CarResponseDto>),
StatusCodes.Status200OK)]
    public async Task<ActionResult<List<CarResponseDto>>> GetAll()
    {
        var cars = await carService.GetAllAsync();
        return Ok(cars);
    }

    [HttpGet("{id:int}")]
    [ProducesResponseType(typeof(CarResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<CarResponseDto>> GetById(int id)
    {
        var car = await carService.GetByIdAsync(id);
        return Ok(car);
    }

    [HttpPost]
    [ProducesResponseType(typeof(CarResponseDto), StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
          public   async   Task<ActionResult<CarResponseDto>>   Create([FromBody]
CarCreateDto dto)
```

```
    {
        var created = await carService.CreateAsync(dto);
            return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
    }

    [HttpPut("{id:int}")]
    [ProducesResponseType(typeof(CarResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
     public async Task<ActionResult<CarResponseDto>> Update(int id, [FromBody]
CarUpdateDto dto)
    {
        var updated = await carService.UpdateAsync(id, dto);
        return Ok(updated);
    }

    [HttpDelete("{id:int}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> Delete(int id)
    {
        await carService.DeleteAsync(id);
        return NoContent();
    }

    [HttpDelete]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    public async Task<IActionResult> DeleteAll()
    {
        await carService.DeleteAllAsync();
        return NoContent();
    }

    [HttpGet("{id:int}/drivers")]
                            [ProducesResponseType(typeof(List<DriverResponseDto>),
StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
                public      async      Task<ActionResult<List<DriverResponseDto>>>
GetDriversByCarId(int id)
    {
        var drivers = await carService.GetDriversByCarIdAsync(id);
        return Ok(drivers);
    }
```

```
}
```

```
using Microsoft.AspNetCore.Mvc;
using Taxi.DTOs;
using Taxi.Services;

namespace Taxi.Controllers;

[ApiController]
[Route("api/[controller]")]
[Produces("application/json")]
public class DriversController : ControllerBase
{
    private readonly IDriverService driverService;

    public DriversController(IDriverService driverService)
    {
        this.driverService = driverService;
    }

    [HttpGet]
                            [ProducesResponseType(typeof(List<DriverResponseDto>),
StatusCodes.Status200OK)]
    public async Task<ActionResult<List<DriverResponseDto>>> GetAll()
    {
        var drivers = await driverService.GetAllAsync();
        return Ok(drivers);
    }

    [HttpGet("{id:int}")]
    [ProducesResponseType(typeof(DriverResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<DriverResponseDto>> GetById(int id)
    {
        var driver = await driverService.GetByIdAsync(id);
        return Ok(driver);
    }

    [HttpPost]
                                  [ProducesResponseType(typeof(DriverResponseDto),
StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
        public   async   Task<ActionResult<DriverResponseDto>>   Create([FromBody]
DriverCreateDto dto)
```

```
    {
        var created = await driverService.CreateAsync(dto);
            return CreatedAtAction(nameof(GetById), new { id = created.Id },
created);
    }

    [HttpPut("{id:int}")]
    [ProducesResponseType(typeof(DriverResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<ActionResult<DriverResponseDto>> Update(int id, [FromBody]
DriverUpdateDto dto)
    {
        var updated = await driverService.UpdateAsync(id, dto);
        return Ok(updated);
    }

    [HttpDelete("{id:int}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> Delete(int id)
    {
        await driverService.DeleteAsync(id);
        return NoContent();
    }

    [HttpDelete]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    public async Task<IActionResult> DeleteAll()
    {
        await driverService.DeleteAllAsync();
        return NoContent();
    }
}
```

Data

```
using Microsoft.EntityFrameworkCore;
using Taxi.Models;


namespace Taxi.Data;


public class TaxiDbContext : DbContext
```

```
{
    public TaxiDbContext(DbContextOptions<TaxiDbContext> options) :
base(options) {  }

    public DbSet<Car> Cars => Set<Car>();

    public DbSet<Driver> Drivers => Set<Driver>();


    protected override void OnModelCreating(ModelBuilder modelBuilder)

    {

        base.OnModelCreating(modelBuilder);


        modelBuilder.Entity<Car>(entity =>

        {

            entity.ToTable("Cars");

            entity.HasKey(c => c.Id);

            entity.Property(c => c.Brand).IsRequired().HasMaxLength(50);

            entity.Property(c => c.Model).IsRequired().HasMaxLength(50);

            entity.Property(c => c.EngineVolume).IsRequired();

            entity.Property(c => c.Year).IsRequired();

        });


        modelBuilder.Entity<Driver>(entity =>

        {

            entity.ToTable("Drivers", t =>
t.HasCheckConstraint("CK_Drivers_Rating", "\"Rating\" >= 0 AND \"Rating\" <=
5"));

            entity.HasKey(d => d.Id);

            entity.Property(d => d.FullName).IsRequired().HasMaxLength(100);

            entity.Property(d => d.Rating).IsRequired();

            entity.Property(d => d.IsBusy).IsRequired();

            entity.HasOne(d => d.Car).WithMany(c => c.Drivers).HasForeignKey(d
=> d.CarId).OnDelete(DeleteBehavior.Cascade);

        });

    }

}
```

DTOs

```
namespace Taxi.DTOs;


public abstract class CarBaseDto
{
    public string Brand { get; set; } = string.Empty;
    public string Model { get; set; } = string.Empty;
    public double EngineVolume { get; set; }
    public int Year { get; set; }
}
```

```
namespace Taxi.DTOs;


public class CarCreateDto : CarBaseDto
{
}
```

```
namespace Taxi.DTOs;


public class CarResponseDto
{
    public int Id { get; set; }
    public string Brand { get; set; } = string.Empty;
    public string Model { get; set; } = string.Empty;
    public double EngineVolume { get; set; }
    public int Year { get; set; }
    public int Age { get; set; }
}
```

```
namespace Taxi.DTOs;


public class CarUpdateDto : CarBaseDto
```

```
{

}
```

```
namespace Taxi.DTOs;


public abstract class DriverBaseDto

{

    public string FullName { get; set; } = string.Empty;

    public double Rating { get; set; }

    public bool IsBusy { get; set; }

    public int CarId { get; set; }

}
```

```
namespace Taxi.DTOs;


public class DriverCreateDto : DriverBaseDto

{

}
```

```
namespace Taxi.DTOs;


public class DriverResponseDto

{

    public int Id { get; set; }

    public string FullName { get; set; } = string.Empty;

    public double Rating { get; set; }

    public bool IsBusy { get; set; }

    public int CarId { get; set; }

}
```

```
namespace Taxi.DTOs;


public class DriverUpdateDto : DriverBaseDto
```

```
{

}
```

Exceptions

```
namespace Taxi.Exceptions;


public class BadRequestException : Exception

{

    public BadRequestException(string message) : base(message)

    {

    }

}
```

```
namespace Taxi.Exceptions;


public class ConflictException : Exception

{

    public ConflictException(string message) : base(message)

    {

    }

}
```

```
namespace Taxi.Exceptions;


public class NotFoundException : Exception

{

    public NotFoundException(string message) : base(message)

    {

    }

}
```

Mappings

```
using AutoMapper;
using Taxi.DTOs;
using Taxi.Models;
using Taxi.Validators;

namespace Taxi.Mappings;

public class MappingProfile : Profile
{
   public MappingProfile()
   {
       CreateMap<Car, CarResponseDto>()
           .ForMember(dest => dest.Age, opt => opt.MapFrom(src =>
DateTime.UtcNow.Year - src.Year));

       CreateMap<CarBaseDto, Car>()
           .ForMember(dest => dest.Id, opt => opt.Ignore())
           .ForMember(dest => dest.Drivers, opt => opt.Ignore())
           .ForMember(dest => dest.Brand, opt => opt.MapFrom(src =>
src.Brand.Trim()))
           .ForMember(dest => dest.Model, opt => opt.MapFrom(src =>
src.Model.Trim()));

       CreateMap<CarCreateDto, Car>().IncludeBase<CarBaseDto, Car>();
       CreateMap<CarUpdateDto, Car>().IncludeBase<CarBaseDto, Car>();

       CreateMap<Driver, DriverResponseDto>();

       CreateMap<DriverBaseDto, Driver>()
           .ForMember(dest => dest.Id, opt => opt.Ignore())
           .ForMember(dest => dest.Car, opt => opt.Ignore())
           .ForMember(dest => dest.FullName, opt => opt.MapFrom(src =>
src.FullName.Trim()));
```

```
        CreateMap<DriverCreateDto, Driver>().IncludeBase<DriverBaseDto,
Driver>();

        CreateMap<DriverUpdateDto, Driver>().IncludeBase<DriverBaseDto,
Driver>();

    }

}
```

Middleware

```
using System.Text.Encodings.Web;

using System.Text.Json;

using System.Text.Json.Serialization;

using System.Text.Unicode;

using FluentValidation;

using Microsoft.EntityFrameworkCore;

using Npgsql;

using Taxi.Exceptions;


namespace Taxi.Middleware;


public class ExceptionHandlingMiddleware

{

    private static readonly JsonSerializerOptions JsonOptions = new()

    {

        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,

        DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,

        Encoder = JavaScriptEncoder.Create(UnicodeRanges.All)

    };


    private readonly RequestDelegate next;

    private readonly ILogger<ExceptionHandlingMiddleware> logger;


    public ExceptionHandlingMiddleware(RequestDelegate next,
ILogger<ExceptionHandlingMiddleware> logger)

    {
```

```
        this.next = next;
        this.logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await next(context);
        }
        catch (Exception ex)
        {
            if (context.Response.HasStarted)
            {
                logger.LogError(ex, "Помилка після початку відправлення
відповіді.");
                throw;
            }

            await HandleExceptionAsync(context, ex);
        }
    }

    private async Task HandleExceptionAsync(HttpContext context, Exception
exception)
    {
        int statusCode;
        string message;
        Dictionary<string, string[]>? errors = null;

        switch (exception)
        {
            case ValidationException validationException:
                statusCode = StatusCodes.Status400BadRequest;
```

```
                message = "Помилка валідації вхідних даних.";
                errors = validationException.Errors.GroupBy(e =>
e.PropertyName).ToDictionary(g => g.Key, g => g.Select(e =>
e.ErrorMessage).Distinct().ToArray());
                break;

            case BadRequestException:
                statusCode = StatusCodes.Status400BadRequest;
                message = exception.Message;
                break;

            case NotFoundException:
                statusCode = StatusCodes.Status404NotFound;
                message = exception.Message;
                break;

            case ConflictException:
                statusCode = StatusCodes.Status409Conflict;
                message = exception.Message;
                break;

            case DbUpdateException { InnerException: PostgresException {
SqlState: PostgresErrorCodes.UniqueViolation } }:
                statusCode = StatusCodes.Status409Conflict;
                message = "Запис з такими унікальними даними вже існує.";
                break;

            default:
                statusCode = StatusCodes.Status500InternalServerError;
                message = "Внутрішня помилка сервера.";
                logger.LogError(exception, "Необроблений виняток під час
виконання запиту {Method} {Path}", context.Request.Method,
context.Request.Path);
                break;
        }
```

```
        context.Response.Clear();
        context.Response.StatusCode = statusCode;
        context.Response.ContentType = "application/json; charset=utf-8";

        var payload = new ErrorResponse(statusCode, message, errors);
        await context.Response.WriteAsync(JsonSerializer.Serialize(payload,
JsonOptions));
    }

    private sealed record ErrorResponse(int Status, string Error,
Dictionary<string, string[]>? Errors);
}
```

Models

```
namespace Taxi.Models;

public class Car
{
    public int Id { get; set; }
    public string Brand { get; set; } = string.Empty;
    public string Model { get; set; } = string.Empty;
    public double EngineVolume { get; set; }
    public int Year { get; set; }
    public ICollection<Driver> Drivers { get; set; } = new List<Driver>();
}
```

```
namespace Taxi.Models;

public class Driver
{
    public int Id { get; set; }
    public string FullName { get; set; } = string.Empty;
```

```
    public double Rating { get; set; }

    public bool IsBusy { get; set; }

    public int CarId { get; set; }

    public Car? Car { get; set; }
}
```

Repositories

```
using Microsoft.EntityFrameworkCore;

using Taxi.Data;

using Taxi.Models;


namespace Taxi.Repositories;


public class CarRepository : ICarRepository

{

    private readonly TaxiDbContext context;


    public CarRepository(TaxiDbContext context)

    {

        this.context = context;

    }


    public async Task<List<Car>> GetAllAsync()

    {

        return await context.Cars.AsNoTracking().OrderBy(c =>
c.Id).ToListAsync();

    }


    public async Task<Car?> GetByIdAsync(int id)

    {

        return await context.Cars.FirstOrDefaultAsync(c => c.Id == id);

    }
```

```
    public async Task<bool> ExistsAsync(int id)
    {
        return await context.Cars.AnyAsync(c => c.Id == id);
    }

    public async Task AddAsync(Car car)
    {
        await context.Cars.AddAsync(car);
        await context.SaveChangesAsync();
    }

    public async Task UpdateAsync(Car car)
    {
        context.Cars.Update(car);
        await context.SaveChangesAsync();
    }

    public async Task DeleteAsync(Car car)
    {
        context.Cars.Remove(car);
        await context.SaveChangesAsync();
    }

    public async Task<int> DeleteAllAsync()
    {
        return await context.Cars.ExecuteDeleteAsync();
    }
}
```

```
using Microsoft.EntityFrameworkCore;
using Taxi.Data;
using Taxi.Models;
```

```
namespace Taxi.Repositories;


public class DriverRepository : IDriverRepository
{
    private readonly TaxiDbContext context;

    public DriverRepository(TaxiDbContext context)
    {
        this.context = context;
    }

    public async Task<List<Driver>> GetAllAsync()
    {
        return await context.Drivers.AsNoTracking().OrderBy(d =>
d.Id).ToListAsync();
    }

    public async Task<Driver?> GetByIdAsync(int id)
    {
        return await context.Drivers.FirstOrDefaultAsync(d => d.Id == id);
    }

    public async Task<List<Driver>> GetByCarIdAsync(int carId)
    {
        return await context.Drivers.AsNoTracking().Where(d => d.CarId ==
carId).OrderBy(d => d.Id).ToListAsync();
    }

    public async Task AddAsync(Driver driver)
    {
        await context.Drivers.AddAsync(driver);
        await context.SaveChangesAsync();
    }
```

```
    public async Task UpdateAsync(Driver driver)
    {
        context.Drivers.Update(driver);
        await context.SaveChangesAsync();
    }

    public async Task DeleteAsync(Driver driver)
    {
        context.Drivers.Remove(driver);
        await context.SaveChangesAsync();
    }

    public async Task<int> DeleteAllAsync()
    {
        return await context.Drivers.ExecuteDeleteAsync();
    }
}
```

```
using Taxi.Models;

namespace Taxi.Repositories;

public interface ICarRepository
{
    Task<List<Car>> GetAllAsync();
    Task<Car?> GetByIdAsync(int id);
    Task<bool> ExistsAsync(int id);
    Task AddAsync(Car car);
    Task UpdateAsync(Car car);
    Task DeleteAsync(Car car);
    Task<int> DeleteAllAsync();
}
using Taxi.Models;
```

```
namespace Taxi.Repositories;


public interface IDriverRepository
{
    Task<List<Driver>> GetAllAsync();
    Task<Driver?> GetByIdAsync(int id);
    Task<List<Driver>> GetByCarIdAsync(int carId);
    Task AddAsync(Driver driver);
    Task UpdateAsync(Driver driver);
    Task DeleteAsync(Driver driver);
    Task<int> DeleteAllAsync();
}
```

Services

```
using AutoMapper;
using FluentValidation;
using Taxi.DTOs;
using Taxi.Exceptions;
using Taxi.Models;
using Taxi.Repositories;


namespace Taxi.Services;


public class CarService : ICarService
{
    private readonly ICarRepository carRepository;
    private readonly IDriverRepository driverRepository;
    private readonly IMapper mapper;
    private readonly IValidator<CarCreateDto> createValidator;
    private readonly IValidator<CarUpdateDto> updateValidator;
    private readonly ILogger<CarService> logger;
```

```
    public CarService(ICarRepository carRepository, IDriverRepository
driverRepository, IMapper mapper, IValidator<CarCreateDto> createValidator,
IValidator<CarUpdateDto> updateValidator, ILogger<CarService> logger)
    {
        this.carRepository = carRepository;
        this.driverRepository = driverRepository;
        this.mapper = mapper;
        this.createValidator = createValidator;
        this.updateValidator = updateValidator;
        this.logger = logger;
    }

    public async Task<List<CarResponseDto>> GetAllAsync()
    {
        logger.LogInformation("Виконано запит на отримання всіх автомобілів.");
        var cars = await carRepository.GetAllAsync();
        return mapper.Map<List<CarResponseDto>>(cars);
    }

    public async Task<CarResponseDto> GetByIdAsync(int id)
    {
        logger.LogInformation("Пошук автомобіля з ID: {Id}", id);
        var car = await GetExistingCarAsync(id);
        logger.LogInformation("Автомобіль знайдено: {Brand} {Model}", car.Brand,
car.Model);
        return mapper.Map<CarResponseDto>(car);
    }

    public async Task<CarResponseDto> CreateAsync(CarCreateDto dto)
    {
        await createValidator.ValidateAndThrowAsync(dto);

        var car = mapper.Map<Car>(dto);
        await carRepository.AddAsync(car);
```

```
        logger.LogInformation("[Успіх] Додано новий автомобіль: {Brand} {Model}, ID: {Id}", car.Brand, car.Model, car.Id);
        return mapper.Map<CarResponseDto>(car);
    }

    public async Task<CarResponseDto> UpdateAsync(int id, CarUpdateDto dto)
    {
        await updateValidator.ValidateAndThrowAsync(dto);

        var car = await GetExistingCarAsync(id);

        mapper.Map(dto, car);
        await carRepository.UpdateAsync(car);

        logger.LogInformation("[Успіх] Автомобіль ID {Id} успішно оновлено.", id);
        return mapper.Map<CarResponseDto>(car);
    }

    public async Task DeleteAsync(int id)
    {
        var car = await GetExistingCarAsync(id);
        await carRepository.DeleteAsync(car);
        logger.LogInformation("[Успіх] Автомобіль ID {Id} та його водіїв видалено з бази.", id);
    }

    public async Task DeleteAllAsync()
    {
        var deleted = await carRepository.DeleteAllAsync();
        logger.LogWarning("[Увага] Видалено всі автомобілі ({Count}) разом з їхніми водіями!", deleted);
    }
```

```
    public async Task<List<DriverResponseDto>> GetDriversByCarIdAsync(int carId)
    {
        if (!await carRepository.ExistsAsync(carId))
        {
            logger.LogWarning("[Увага] Спроба знайти водіїв для неіснуючого авто (ID {Id})", carId);
            throw new NotFoundException($"Автомобіль з ID {carId} не знайдено.");
        }

        var drivers = await driverRepository.GetByCarIdAsync(carId);
        return mapper.Map<List<DriverResponseDto>>(drivers);
    }

    private async Task<Car> GetExistingCarAsync(int id)
    {
        var car = await carRepository.GetByIdAsync(id);
        if (car is null)
        {
            logger.LogWarning("[Увага] Автомобіль з ID {Id} не знайдено в базі!", id);
            throw new NotFoundException($"Автомобіль з ID {id} не знайдено.");
        }

        return car;
    }
}
```

```
using AutoMapper;
using FluentValidation;
using Taxi.DTOs;
using Taxi.Exceptions;
using Taxi.Models;
```

```
using Taxi.Repositories;


namespace Taxi.Services;


public class DriverService : IDriverService
{
    private readonly IDriverRepository driverRepository;
    private readonly ICarRepository carRepository;
    private readonly IMapper mapper;
    private readonly IValidator<DriverCreateDto> createValidator;
    private readonly IValidator<DriverUpdateDto> updateValidator;
    private readonly ILogger<DriverService> logger;


    public DriverService(IDriverRepository driverRepository, ICarRepository
carRepository, IMapper mapper, IValidator<DriverCreateDto> createValidator,
IValidator<DriverUpdateDto> updateValidator, ILogger<DriverService> logger)
    {
        this.driverRepository = driverRepository;
        this.carRepository = carRepository;
        this.mapper = mapper;
        this.createValidator = createValidator;
        this.updateValidator = updateValidator;
        this.logger = logger;
    }


    public async Task<List<DriverResponseDto>> GetAllAsync()
    {
        logger.LogInformation("Виконано запит на отримання всіх водіїв.");
        var drivers = await driverRepository.GetAllAsync();
        return mapper.Map<List<DriverResponseDto>>(drivers);
    }


    public async Task<DriverResponseDto> GetByIdAsync(int id)
    {
```

```
        logger.LogInformation("Пошук водія з ID: {Id}", id);
        var driver = await GetExistingDriverAsync(id);
        logger.LogInformation("Водія знайдено: {FullName}", driver.FullName);
        return mapper.Map<DriverResponseDto>(driver);
    }

    public async Task<DriverResponseDto> CreateAsync(DriverCreateDto dto)
    {
        await createValidator.ValidateAndThrowAsync(dto);
        await EnsureCarExistsAsync(dto.CarId);

        var driver = mapper.Map<Driver>(dto);
        await driverRepository.AddAsync(driver);

        logger.LogInformation("[Успіх] Додано нового водія: {FullName}, ID:
{Id}", driver.FullName, driver.Id);
        return mapper.Map<DriverResponseDto>(driver);
    }

    public async Task<DriverResponseDto> UpdateAsync(int id, DriverUpdateDto
dto)
    {
        await updateValidator.ValidateAndThrowAsync(dto);

        var driver = await GetExistingDriverAsync(id);

        if (driver.CarId != dto.CarId)
        {
            await EnsureCarExistsAsync(dto.CarId);
        }

        mapper.Map(dto, driver);
        await driverRepository.UpdateAsync(driver);
```

```
        logger.LogInformation("[Успіх] Дані водія ID {Id} успішно оновлено.",
id);

        return mapper.Map<DriverResponseDto>(driver);

    }


    public async Task DeleteAsync(int id)

    {

        var driver = await GetExistingDriverAsync(id);

        await driverRepository.DeleteAsync(driver);

        logger.LogInformation("[Успіх] Водія ID {Id} видалено з бази.", id);

    }


    public async Task DeleteAllAsync()

    {

        var deleted = await driverRepository.DeleteAllAsync();

        logger.LogWarning("[Увага] Видалено всіх водіїв ({Count})!", deleted);

    }


    private async Task<Driver> GetExistingDriverAsync(int id)

    {

        var driver = await driverRepository.GetByIdAsync(id);

        if (driver is null)

        {

            logger.LogWarning("[Увага] Водія з ID {Id} не знайдено в базі!",
id);

            throw new NotFoundException($"Водія з ID {id} не знайдено.");

        }


        return driver;

    }


    private async Task EnsureCarExistsAsync(int carId)

    {

        if (!await carRepository.ExistsAsync(carId))
```

```
        {
            logger.LogWarning("[Помилка] Спроба прив'язати водія до неіснуючого авто (CarId {CarId})", carId);
            throw new BadRequestException($"Автомобіль з ID {carId} не існує.");
        }
    }
}
```

```
using Taxi.DTOs;

namespace Taxi.Services;

public interface ICarService
{
    Task<List<CarResponseDto>> GetAllAsync();
    Task<CarResponseDto> GetByIdAsync(int id);
    Task<CarResponseDto> CreateAsync(CarCreateDto dto);
    Task<CarResponseDto> UpdateAsync(int id, CarUpdateDto dto);
    Task DeleteAsync(int id);
    Task DeleteAllAsync();
    Task<List<DriverResponseDto>> GetDriversByCarIdAsync(int carId);
}
```

```
using Taxi.DTOs;

namespace Taxi.Services;

public interface IDriverService
{
    Task<List<DriverResponseDto>> GetAllAsync();
    Task<DriverResponseDto> GetByIdAsync(int id);
    Task<DriverResponseDto> CreateAsync(DriverCreateDto dto);
    Task<DriverResponseDto> UpdateAsync(int id, DriverUpdateDto dto);
```

```
    Task DeleteAsync(int id);

    Task DeleteAllAsync();

}
```

Validators

```
using FluentValidation;

using Taxi.DTOs;


namespace Taxi.Validators;


public abstract class CarBaseDtoValidator<T> : AbstractValidator<T> where T :
CarBaseDto

{

    protected CarBaseDtoValidator()

    {

        RuleLevelCascadeMode = CascadeMode.Stop;


        var maxYear = DateTime.UtcNow.Year + 1;


        RuleFor(x => x.Brand)

            .NotEmpty().WithMessage("Марка автомобіля є обов'язковою.")

            .MaximumLength(50).WithMessage("Марка автомобіля не може
перевищувати 50 символів.");


        RuleFor(x => x.Model)

            .NotEmpty().WithMessage("Модель автомобіля є обов'язковою.")

            .MaximumLength(50).WithMessage("Модель автомобіля не може
перевищувати 50 символів.");


        RuleFor(x => x.EngineVolume)

            .InclusiveBetween(0.0, 10.0).WithMessage("Об'єм двигуна має бути від
0 до 10 л (0 — для електромобілів).");


        RuleFor(x => x.Year)
```

```
            .InclusiveBetween(ValidationRules.MinCarYear,
maxYear).WithMessage($"Рік випуску має бути в межах від
{ValidationRules.MinCarYear} до {maxYear}.");

    }

}
```

```
using Taxi.DTOs;


namespace Taxi.Validators;


public class CarCreateDtoValidator : CarBaseDtoValidator<CarCreateDto>

{

}
```

```
using Taxi.DTOs;


namespace Taxi.Validators;


public class CarUpdateDtoValidator : CarBaseDtoValidator<CarUpdateDto>

{

}
```

```
using FluentValidation;

using Taxi.DTOs;


namespace Taxi.Validators;


public abstract class DriverBaseDtoValidator<T> : AbstractValidator<T> where T
: DriverBaseDto

{

    protected DriverBaseDtoValidator()

    {

        RuleLevelCascadeMode = CascadeMode.Stop;
```

```
        RuleFor(x => x.FullName)
            .NotEmpty().WithMessage("ПІБ водія є обов'язковим.")
            .MinimumLength(3).WithMessage("ПІБ водія має містити щонайменше 3 символи.")
            .MaximumLength(100).WithMessage("ПІБ водія не може перевищувати 100 символів.")
            .Must(ValidationRules.IsValidPersonName).WithMessage("ПІБ може містити лише літери, пробіли, дефіс та апостроф.");

        RuleFor(x => x.Rating)
            .InclusiveBetween(0.0, 5.0).WithMessage("Рейтинг водія має бути в межах від 0 до 5.");

        RuleFor(x => x.CarId)
            .GreaterThan(0).WithMessage("Ідентифікатор автомобіля (CarId) має бути додатним числом.");
    }
}
```

```
using Taxi.DTOs;

namespace Taxi.Validators;

public class DriverCreateDtoValidator : DriverBaseDtoValidator<DriverCreateDto>
{
}
```

```
using Taxi.DTOs;

namespace Taxi.Validators;

public class DriverUpdateDtoValidator : DriverBaseDtoValidator<DriverUpdateDto>
{
}
```

```
using System.Text.RegularExpressions;


namespace Taxi.Validators;


public static class ValidationRules
{
    public const int MinCarYear = 2000;


    private static readonly Regex PersonNameRegex =
        new(@"^\p{L}+(?:[ ''\-]\p{L}+)*$", RegexOptions.Compiled);


    public static bool IsValidPersonName(string? value) =>
!string.IsNullOrWhiteSpace(value) && PersonNameRegex.IsMatch(value.Trim());
}
```

Program

```
using System.Text.Encodings.Web;
using System.Text.Unicode;
using AutoMapper;
using FluentValidation;
using Microsoft.EntityFrameworkCore;
using Taxi.Data;
using Taxi.Mappings;
using Taxi.Middleware;
using Taxi.Repositories;
using Taxi.Services;
using Taxi.Validators;


var builder = WebApplication.CreateBuilder(args);


builder.Services
    .AddControllers(options =>
    {
```

```
        options.SuppressImplicitRequiredAttributeForNonNullableReferenceTypes =
true;
    })
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.Encoder =
JavaScriptEncoder.Create(UnicodeRanges.All);
    });


builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();



var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("Рядок підключення
'DefaultConnection' не знайдено.");


builder.Services.AddDbContext<TaxiDbContext>(options =>
options.UseNpgsql(connectionString));
builder.Services.AddScoped<ICarRepository, CarRepository>();
builder.Services.AddScoped<IDriverRepository, DriverRepository>();
builder.Services.AddScoped<ICarService, CarService>();
builder.Services.AddScoped<IDriverService, DriverService>();
builder.Services.AddValidatorsFromAssemblyContaining<CarCreateDtoValidator>();
builder.Services.AddAutoMapper(typeof(MappingProfile));


var app = builder.Build();
app.UseMiddleware<ExceptionHandlingMiddleware>();


if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
```

```
app.Services.GetRequiredService<IMapper>().ConfigurationProvider.AssertConfigur
ationIsValid();


    await using var scope = app.Services.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<TaxiDbContext>();


    try
    {
        if (db.Database.GetMigrations().Any())
        {
            await db.Database.MigrateAsync();
        }
        else
        {
            app.Logger.LogWarning(
                "Міграцій не знайдено. Виконайте 'dotnet ef migrations add
InitialCreate' та 'dotnet ef database update'.");
        }
    }
    catch (Exception ex)
    {
        app.Logger.LogError(ex, "Не вдалося підключитися до PostgreSQL або
застосувати міграції. Перевірте рядок підключення.");
    }
}


app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();


await app.RunAsync();
```

Знімки екрану з прикладами результатів роботи валідації DTO

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7043/api/Cars' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "brand": "Toyota",
  "model": "Camry",
  "engineVolume": 2.5,
  "year": 2021
}'
```

Request URL

```
https://localhost:7043/api/Cars
```

Server response

Code	Details
201	Response body

```
{
  "id": 1,
  "brand": "Toyota",
  "model": "Camry",
  "engineVolume": 2.5,
  "year": 2021,
  "age": 5
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,29 Oct 2026 00:52:02 GMT
location: https://localhost:7043/api/Cars/1
server: Kestrel
```

Responses

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7043/api/Cars' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "brand": "",
  "model": "Camry",
  "engineVolume": 2.5,
  "year": 1985
}'
```

Request URL

```
https://localhost:7043/api/Cars
```

Server response

Code	Details
400	Error: response status is 400

Response body

```
{
  "status": 400,
  "error": "Помилка валідації вхідних даних.",
  "errors": {
    "Brand": [
      "Марка автомобіля є обов'язковою."
    ],
    "Year": [
      "Рік випуску має бути в межах від 2000 до 2027."
    ]
  }
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,29 Oct 2026 00:52:34 GMT
server: Kestrel
```

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7043/api/Drivers' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "fullName": "Богдан Жижка",
  "rating": 4.9,
  "isBusy": false,
  "carId": 1
}'
```

Request URL

```
https://localhost:7043/api/Drivers
```

Server response

Code	Details
201	Response body

```
{
  "id": 1,
  "fullName": "Богдан Жижка",
  "rating": 4.9,
  "isBusy": false,
  "carId": 1
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,29 Oct 2026 00:53:18 GMT
location: https://localhost:7043/api/Drivers/1
server: Kestrel
```

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7043/api/Drivers' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "fullName": "R2-D2",
  "rating": 7.5,
  "isBusy": false,
  "carId": 1
}'
```

Request URL

```
https://localhost:7043/api/Drivers
```

Server response

Code	Details
400	Error: response status is 400

Response body

```
{
  "status": 400,
  "error": "Помилка валідації вхідних даних.",
  "errors": {
    "FullName": [
      "ПІБ може містити лише літери, пробіли, дефіс та апостроф."
    ],
    "Rating": [
      "Рейтинг водія має бути в межах від 0 до 5."
    ]
  }
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,29 Oct 2026 00:54:03 GMT
server: Kestrel
```

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7043/api/Drivers' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "fullName": "Іван Петренко",
  "rating": 4.0,
  "isBusy": false,
  "carId": 999
}'
```

Request URL

```
https://localhost:7043/api/Drivers
```

Server response

Code	Details
400	Error: response status is 400

Response body

```
{
  "status": 400,
  "error": "Автомобіль з ID 999 не існує."
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,29 Oct 2026 00:54:39 GMT
server: Kestrel
```

Responses

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7043/api/Cars/999/drivers' \
  -H 'accept: application/json'
```

Request URL

```
https://localhost:7043/api/Cars/999/drivers
```

Server response

Code	Details
404	Error: response status is 404

Response body

```
{
  "status": 404,
  "error": "Автомобіль з ID 999 не знайдено."
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,29 Oct 2026 00:55:17 GMT
server: Kestrel
```

Responses

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7043/api/Cars' \
  -H 'accept: application/json'
```

Request URL

```
https://localhost:7043/api/Cars
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "brand": "Toyota",
    "model": "Camry",
    "engineVolume": 2.5,
    "year": 2021,
    "age": 5
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,29 Oct 2026 00:55:34 GMT
server: Kestrel
```

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7043/api/Drivers' \
  -H 'accept: application/json'
```

Request URL

```
https://localhost:7043/api/Drivers
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "fullName": "Богдан Жижка",
    "rating": 4.9,
    "isBusy": false,
    "carId": 1
  }
]
```

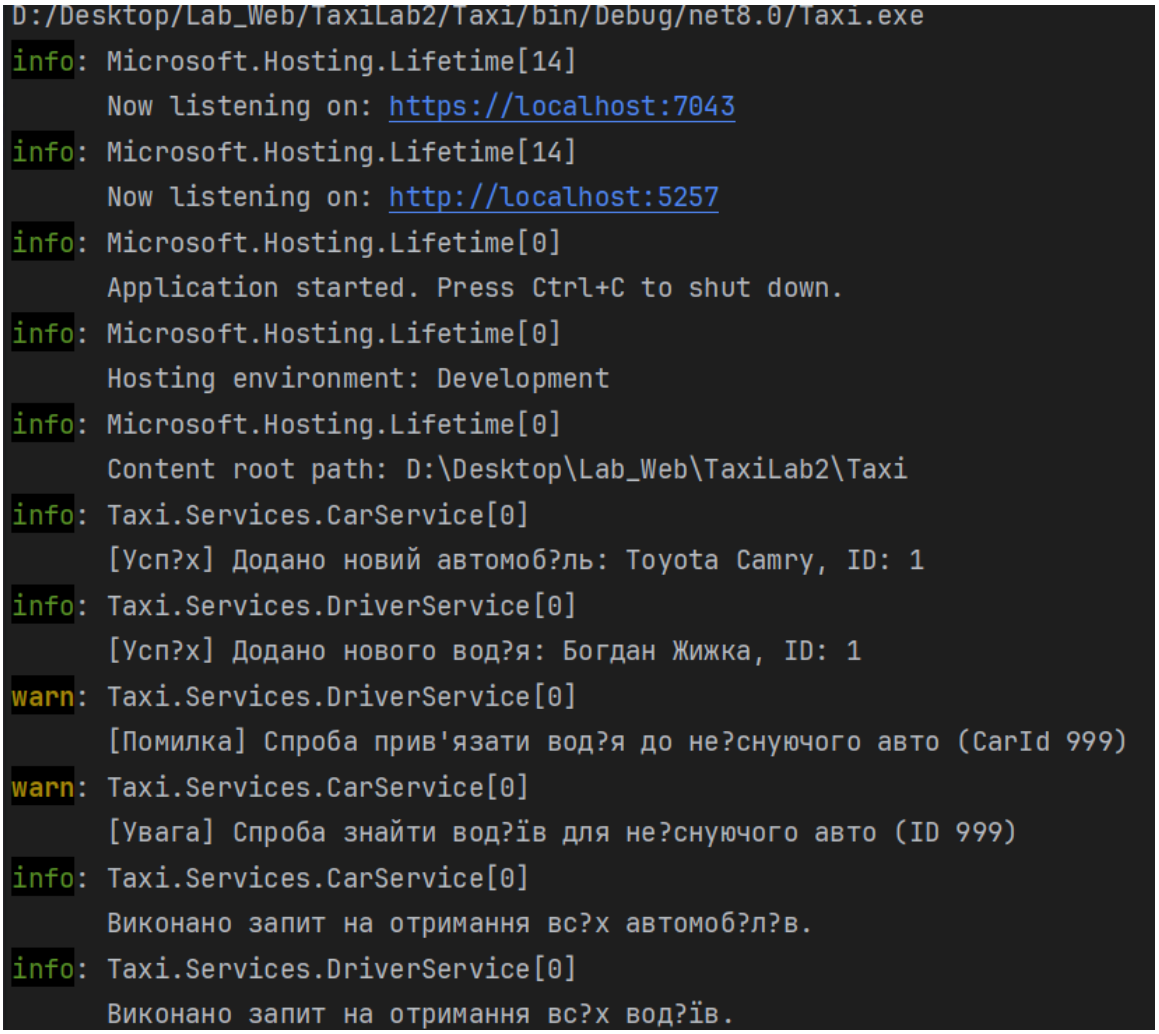
Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu,29 Oct 2026 00:55:57 GMT
server: Kestrel
```

	Id [PK] integer	Brand character varying (50)	Model character varying (50)	EngineVolume double precision	Year integer
1	1	Toyota	Camry	2.5	2021
2	2	Ford	Mondeo	2.5	2002

	Id [PK] integer	FullName character varying (100)	Rating double precision	IsBusy boolean	CarId integer
1	1	Богдан Жижка	4.9	false	1



Висновок

Під час виконання комп’ютерного практикуму програмне забезпечення системи управління таксопарком було переведено з тимчасового зберігання даних у JSON-файлі на постійне зберігання в базі даних PostgreSQL. Для роботи з БД я використав ORM Entity Framework Core та шаблон «Репозиторій», а всі залежності зареєстрував в IoC-контейнері. REST API тепер працює з окремими DTO замість сутностей бази даних, а вхідні DTO перевіряються за допомогою FluentValidation. Додатково я реалізував асинхронні методи роботи з базою даних та автоматичний мапінг сутностей у DTO за допомогою AutoMapper.